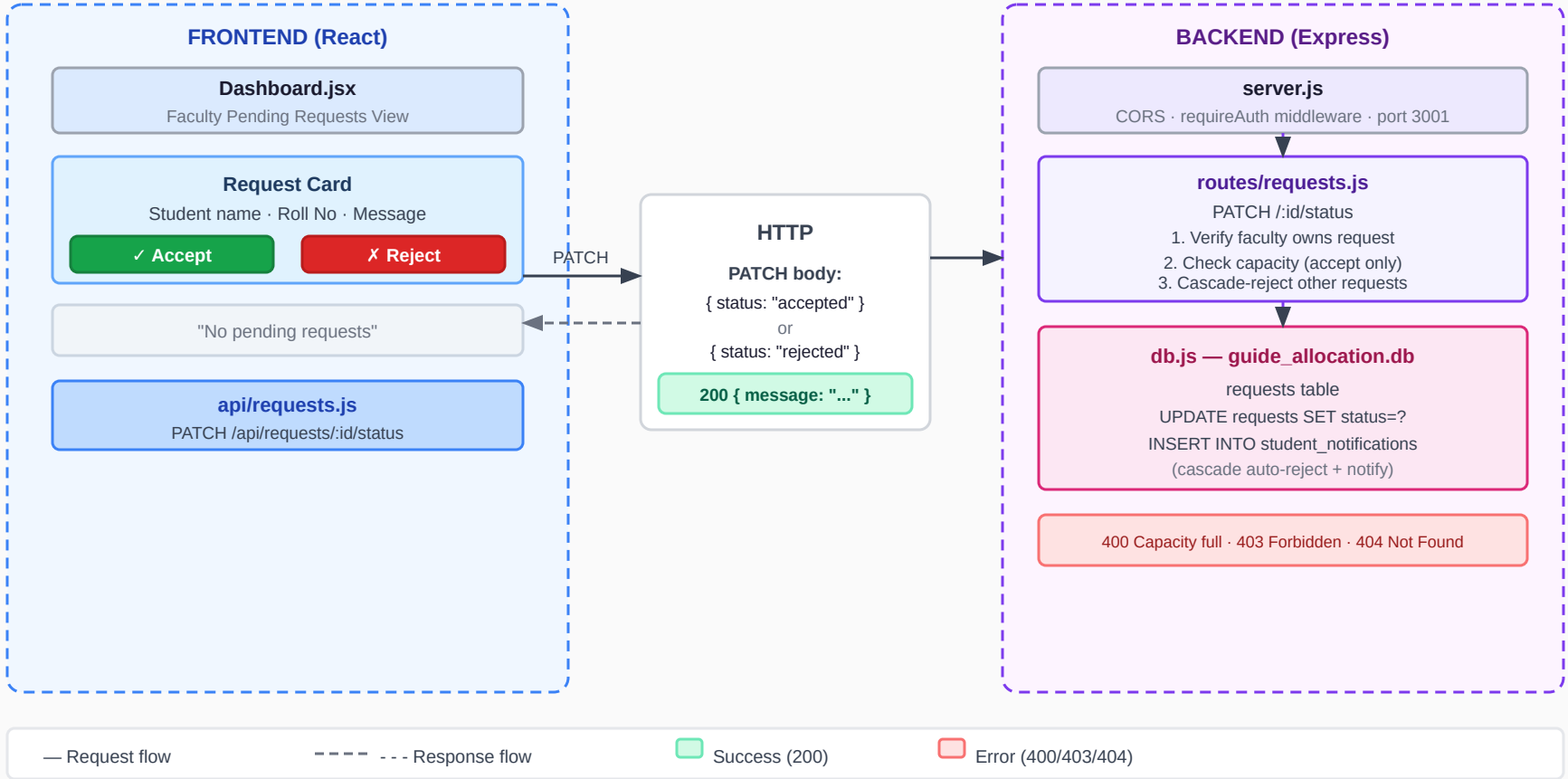


US7 — Faculty Approve or Reject Requests

Architecture Wireframe



GUIDE ALLOCATION SYSTEM — SPRINT 2 · FUNCTIONAL DOCUMENT

US7 — Faculty Approve or Reject Requests

Document ID	GAS-FD-US7	Version	1.0
Prepared By	Development Team	Date	April 2025
User Story	As a faculty member, I want to approve or reject student requests so that only relevant projects are assigned to me.		

1. User Story

US7 — Faculty Approve or Reject Requests

"As a faculty member, I want to approve or reject student project requests so that only relevant projects are assigned to me."

Acceptance Criteria:

- Given a pending request, when the faculty clicks Accept, then the student should be assigned to that faculty and all other pending requests from that student auto-rejected.
- Given a pending request, when the faculty clicks Reject, then the student should be notified and the request closed.

2. Functional Description

Faculty log in to their dashboard and see a *Pending Requests* section listing each student who has requested them as a guide, along with the student's name, roll number, and optional message. Faculty click **Accept** to allocate the student to themselves or **Reject** to decline. On acceptance the backend automatically rejects all other pending requests the student had sent to other faculty members, ensuring one-student-one-guide integrity. Both actions generate an in-app notification for the affected student.

3. Input / Trigger

Field / Action	Type	Required	Constraint
Request ID	Path parameter (URL)	Required	Must belong to a request addressed to the logged-in faculty
status	String (request body)	Required	Must be exactly 'accepted' or 'rejected'
JWT token	Authorization header	Required	Faculty role; 24 h expiry

4. Business Rules

- Faculty can only act on requests where `faculty_id = their own id` ; any other request returns 403.
- Only requests with `status = 'pending'` can be accepted or rejected; already-actioned requests return 400.
- On **Accept**: all other pending requests from the same student are automatically set to `'rejected'` ; the student receives separate notifications for each auto-rejection.
- On **Accept**: the student's `preferred_faculty_id` is updated to this faculty.
- Accept is blocked (400) if the faculty has already reached `max_teams` accepted students.
- On **Reject**: no cascade effect; only the targeted request changes status.
- Both accept and reject insert a record into `student_notifications` for the affected student.
- A faculty member cannot undo an accepted or rejected decision.

5. Process Flow

#	Step	Actor	Details
1	Faculty logs in	Faculty	Authenticated via JWT; role verified as 'faculty'
2	Dashboard loads pending requests	Frontend	GET /api/requests/incoming — returns all requests for this faculty with student details
3	Faculty reviews request card	Faculty	Views student name, roll number, interests/message
4	Faculty clicks Accept or Reject	Faculty	Triggers API call
5	Frontend sends PATCH	Frontend	PATCH /api/requests/:id/status body: { status: 'accepted' } or { status: 'rejected' }
6	Backend validates	Backend	Checks: faculty owns request, status is 'pending', capacity not exceeded (for accept)

#	Step	Actor	Details
7a (Accept)	Cascade reject	Backend / DB	UPDATE requests SET status='rejected' for all other pending requests from same student; insert notification per auto-rejection
7b (Accept)	Update preference	Backend / DB	UPDATE students SET preferred_faculty_id = ? WHERE id = student_id
8	Update request status	Backend / DB	UPDATE requests SET status = 'accepted'/'rejected' WHERE id = ?
9	Notify student	Backend / DB	INSERT INTO student_notifications — 'Your request to Prof. X was accepted/not accepted'
10	Response returned	Backend	200 { message: 'Request accepted' } or { message: 'Request rejected' }
11	UI updates	Frontend	Request card removed from pending list; guide list refreshes to reflect new slot count

6. API Specification

Property	Value
Method	PATCH
URL	/api/requests/:id/status
Auth	Bearer JWT (faculty role)
Content-Type	application/json
Request Body	{ "status": "accepted" } or { "status": "rejected" }
200 — Success	{ "message": "Request accepted" } or { "message": "Request rejected" }
400 — Bad status value	{ "error": "status must be accepted or rejected" }
400 — Not pending	{ "error": "Only pending requests can be updated" }
400 — Capacity full	{ "error": "You have reached your maximum team capacity" }
403 — Wrong faculty	{ "error": "Forbidden" }

Property	Value
404 — Not found	<code>{ "error": "Request not found" }</code>

7. UI Behaviour

Scenario	UI Response
Pending request exists	Card shows student name, roll number, interests, Accept (green) and Reject (red) buttons
Faculty clicks Accept (within capacity)	Card disappears; success state; available-slot count on guide card decrements
Faculty clicks Accept (at max capacity)	Accept button disabled with tooltip 'Maximum students reached'; no API call made
Faculty clicks Reject	Card disappears; no other visible change
No pending requests	Empty state: 'No pending requests at this time'
API error	Toast/inline error message shown; card remains visible

8. Database Schema

requests table

Column	Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
student_id	INTEGER	NOT NULL, FOREIGN KEY → students(id)
faculty_id	INTEGER	NOT NULL, FOREIGN KEY → faculty(id)
status	TEXT	DEFAULT 'pending' — values: pending / accepted / rejected
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

student_notifications table

Column	Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT

Column	Type	Constraint
student_id	INTEGER	NOT NULL, FOREIGN KEY → students(id)
message	TEXT	NOT NULL
read	INTEGER	DEFAULT 0 (0 = unread, 1 = read)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

US7 — Faculty Approve or Reject Requests

Document ID	GAS-TC-US7	Version	1.0
Prepared By	Development Team	Date	April 2025
Environment	Frontend: localhost:5173 · Backend: localhost:3001 · DB: SQLite (guide_allocation.db)		
Precondition	Both servers running; at least one approved faculty with pending student requests in DB		

Test Summary

6 Total	6 Pass	0 Fail	0 Pending
------------	-----------	-----------	--------------

Test Cases

TC ID	Test Case	Priority	Preconditions	Test Steps	Expected Result	Status
TC-US7-01	Faculty accepts a pending request within capacity	High	Faculty logged in; pending request exists; faculty has remaining capacity	1. Navigate to faculty dashboard 2. Locate pending request card 3. Click Accept	Request status → 'accepted'; student receives notification; all other pending requests from same student auto-	PASS

TC ID	Test Case	Priority	Preconditions	Test Steps	Expected Result	Status
					rejected; faculty slot count decrements	
TC-US7-02	Auto-cascade reject on acceptance	High	Same student has pending requests to Faculty A and Faculty B; Faculty A accepts	1. Login as Faculty A 2. Accept the student's request 3. Login as Faculty B and check pending list	Student's request to Faculty B changes to 'rejected'; student receives auto-rejection notification for Faculty B	PASS
TC-US7-03	Faculty rejects a pending request	High	Faculty logged in; pending request exists	1. Navigate to faculty dashboard 2. Click Reject on request card	Request status → 'rejected'; student receives rejection notification; no cascade effect on other requests	PASS
TC-US7-04	Accept blocked when faculty at max capacity	High	Faculty has accepted students count == max_teams; new pending request exists	1. Login as faculty at full capacity 2. View pending request 3. Attempt to accept	400 error returned; UI shows 'Maximum students reached'; request remains pending	PASS
TC-US7-05	Faculty cannot act on another	Medium	Request belongs to Faculty B; Faculty A is logged in	1. Login as Faculty A 2. Send PATCH /api/	403 Forbidden; request	PASS

TC ID	Test Case	Priority	Preconditions	Test Steps	Expected Result	Status
	faculty's request			requests/:id/ status with Faculty B's request ID	status unchanged	
TC- US7-06	Cannot re-act on already accepted/rejected request	Medium	Request with status='accepted' exists	1. Faculty sends PATCH to already-accepted request	400 error: 'Only pending requests can be updated'; no DB change	PASS